alfe – autolisp-front-end – Comprehensive specification

Pascal J. Bourguignon

May 24, 2026

Contents

1 Purpose and scope

This document is the **reference specification** of the `alfe` program (*AutoLISP Front-End*): an orchestration executable that evaluates AutoLISP code either inside a real AutoCAD or BricsCAD instance, or in-process via the AutoLISP runtime provided by `clautolisp`.

`alfe` is the Common Lisp rewrite of the historical bash wrapper `autolisp-script` (whose binary was named `autolisp`), built on top of the already-available `clautolisp/autolisp-*` modules. Every historical reference to "`autolisp`" in this document now designates the `alfe` binary. The public names exposed to user AutoLISP code (global variables `*AUTOLISP-*`, helpers `autolisp-`, environment variables `AUTOLISP_`) are preserved verbatim for backward compatibility with existing scripts.

2 Architecture: alfe clautolisp CAD

2.1 Why a common front-end

BricsCAD and AutoCAD are graphical applications that expose an AutoLISP REPL **behind their GUI**; neither offers a native command-line interface nor any IPC mechanism beyond the file system (AutoLISP itself has neither sockets, nor pipes, nor shared memory). `clautolisp`, by contrast, presents the AutoLISP REPL on the command line and allows every IPC mechanism available to Common Lisp (sockets, named pipes, shared memory, in-process calls via ASDF).

`alfe` is **the** CLI front-end that unifies these two worlds: a single REPL, identical options, identical statuses, regardless of the actual engine.

2.2 Three backend families, two interaction contracts

Backend	Interaction type	Rationale
`clautolisp` (in-process)	direct CL API	`alfe` loads the ASDF `clautolisp/*` packages and
`clautolisp` (subprocess)	stdin/stdout pipes	`alfe` launches `clautolisp-sbcl` and talks to it lin
`bricscad` / `autocad`	File-based protocol	The only IPC available on the CAD side: `alfe` inje runtime that reads `stdin.txt` and writes `stdout.t` `WORKDIR/protocol/`.

The file-based protocol detailed in section Remote REPL protocol is **entirely forced** by the absence of any IPC other than files in the CADs' AutoLISP. On the `clautolisp` side, `alfe` uses the APIs directly and never

touches the file protocol; the workdir exists for symmetry (logs, `--dry-run`) but no communication passes through it.

2.3 Unified execution model

Regardless of the backend, `alfe` exposes the same sequence:

1. Argument parsing and resolution of the effective backend.
2. Creation of the `WORKDIR` (`$AUTOLISP_WORKDIR` or `$TMPDIR/autolisp-runs/<id>`).
3. Backend bring-up:
   - `clautolisp`: direct binding of the evaluation context, or `uiop:launch-program` for the subprocess variant;
   - `bricscad` / `autocad`: emission of `run-common.lsp`, launch of the CAD engine (batch or COM/AppleScript), handshake via `status.txt`.
4. Execution of actions (`-l FILE`, `-x EXPR`, `--main FN`), optional `--interactive`;
5. Status collection, finalisation, cleanup.

Anything not essential for a given backend is disabled for the others: `alfe --clautolisp` does not create a `protocol/` subdirectory, does not emit a CAD-side `run-common.lsp`, does not wait for textual statuses.

2.4 Practical consequences

- User code written once for `alfe` runs on all three backends; residual differences are documented by per-backend constraints (for instance `vlisp-compile` remains exclusive to `--autocad --mode automation`).
- The `clautolisp` and `autolisp-test` conformance suites are usable as-is by `alfe` via the `clautolisp` backend.
- Porting the remote REPL protocol on the CAD side amounts to a CL rewrite of the historical shell driver (`lib/autolisp-remote-io.sh`); but the CAD-side AutoLISP runtime (`runtime/autolisp-remote-io.lsp`) is kept *verbatim*: it is LISP that runs inside the CAD, not CL.

3 AutoLISP runtime-side contract

This section describes the runtime injected on the CAD side (`run-common.lsp` + helpers) and the public values that user code can consume. The contract is identical across the three backends, except that the `clautolisp` backend does not inject anything the `autolisp-` helpers are provided directly by the `autolisp-front-end` subproject's ASDF packages, and the global variables are bound into the evaluation context at startup.

3.1 Injected global variables

The runtime generated in `run-common.lsp` exposes the following variables at the time actions and `MAIN` are executed:

Variable	Role
`*AUTOLISP-VERSION*`	Wrapper version, as `(MAJOR MINOR DEVELOP)`. **New**; resol
`*AUTOLISP-OUTFILE*`	Path to `OUTFILE`.
`*AUTOLISP-ERRFILE*`	Path to `ERRFILE`.
`*AUTOLISP-STATUSFILE*`	Path to `STATUSFILE`.
`*AUTOLISP-INPFILE*`	Path to `INPFILE` (historical REPL).
`*AUTOLISP-LOGDIR*`	Path to the `logs/` directory.
`*AUTOLISP-RENDERFILE*`	Path to the intermediate render file.
`*AUTOLISP-LOGNAME*`	CAD-log filename prefix (default `autolisp-session.log`)
`*AUTOLISP-QUIT-ON-FINISH*`	`T` if the runtime must shut the CAD engine down at the en
`*AUTOLISP-BOOTSTRAP-PHASE*`	Effective bootstrap phase (`marker` / `core` / `log` / `full`).
`*AUTOLISP-INVOCATION-DIR*`	Absolute path of the current directory at wrapper invocati
`*AUTOLISP-DEBUG*`	`T` if `--debug` is active, `nil` otherwise.
`*AUTOLISP-DEBUGFILE*`	Path to `debug.log`.
`*AUTOLISP-USE-REMOTE-PROTOCOL*`	`1` if the remote REPL protocol is active, `0` otherwise.
`*AUTOLISP-PROTOCOL-DIR*`	Path to the `protocol/` subdirectory.
`*AUTOLISP-PROTOCOL-STATUSFILE*`	`protocol/status.txt`.
`*AUTOLISP-PROTOCOL-STDINFILE*`	`protocol/stdin.txt`.
`*AUTOLISP-PROTOCOL-STDOUTFILE*`	`protocol/stdout.txt`.
`*AUTOLISP-PROTOCOL-STDERRFILE*`	`protocol/stderr.txt`.
`*AUTOLISP-PROTOCOL-CONTROLFILE*`	`protocol/control.txt`.
`*AUTOLISP-PROTOCOL-HEARTBEATFILE*`	`protocol/heartbeat.txt`.
`*AUTOLISP-PROTOCOL-READFILE*`	`protocol/read-buffer.lsp`.
`*AUTOLISP-PROTOCOL-INFOFILE*`	`protocol/runtime-info.txt`.
`*AUTOLISP-PROTOCOL-RUNTIMEFILE*`	Path to the protocol's AutoLISP runtime (`runtime/autol`

3.2 Exposed helpers

Function	Role
`(autolisp-log-out string)`	Log a line into `OUTFILE`.
`(autolisp-log-err string)`	Log a line into `ERRFILE`.
`(autolisp-set-status int)`	Write a numeric status into `STATUSFILE`. The wrapper will re
`(autolisp-set-status-text text)`	Write a textual status into `STATUSFILE` (used by the protoco
`(autolisp-resolve-path path)`	Resolve `path` relative to `*AUTOLISP-INVOCATION-DIR*` if it is
`(autolisp-exit)`	Alias of `(exit)`: leave the REPL without killing the engine.
`(quit)`	Request engine shutdown; raises the `*AUTOLISP-QUIT-SIGNA`
`(autolisp-runtime-debug msg)`	Emit a line into `debug.log` when `*AUTOLISP-DEBUG*` is `T`.

3.3 Minimal example

```
(defun C:MAIN ( / )
  (setq *error*
        (lambda (msg)
          (autolisp-log-err (strcat "ERROR: " msg))
          (autolisp-set-status 1)
          (princ)))

  (autolisp-log-out "Hello from AutoLISP.")
  ;; ... your code ...
  (autolisp-set-status 0)
  (princ))
```

3.4 Bootstrap phases

Phase	Normative behaviour
`marker`	Writes only a startup marker and a status. Actions are not executed.
`core`	Executes actions with the basic file I/O; no capture of `print=/=princ=/=prompt`.
`log`	Like `core`, plus the CAD-log setup (`LOGFILEMODE`, `LOGFILEPATH`, `LOGFILENAME`).
`full`	Normal behaviour: CAD log, capture of Lisp outputs, batch REPL if `-i`, remote protocol i

The `--interactive` mode forces `full`; any other combination exits with code 2.

3.5 Emitted `run-common.lsp` template

The template below is the one emitted for a `--bricscad --mode batch -i` session in phase `full`. The `${VAR}` placeholders are substituted by the wrapper. The actual content is the aggregation of several blocks emitted successively by the current bash wrapper (cf. `autolisp:24093563`); the portions elided with `;;` correspond to larger helpers whose details appear in the reference code at the cited lines.

```
;; --- Injected global variables (autolisp:2466-2488) ---
(setq _autolisp_old_srchpath nil)
(setq *AUTOLISP-OUTFILE*                "${LISP_OUTFILE}")
(setq *AUTOLISP-ERRFILE*                "${LISP_ERRFILE}")
(setq *AUTOLISP-STATUSFILE*             "${LISP_STATUSFILE}")
(setq *AUTOLISP-INPFILE*                "${LISP_INPFILE}")
(setq *AUTOLISP-LOGDIR*                 "${LISP_LOGDIR}")
(setq *AUTOLISP-RENDERFILE*             "${LISP_RENDERFILE}")
(setq *AUTOLISP-PROTOCOL-DIR*           "${LISP_PROTOCOL_DIR}")
(setq *AUTOLISP-PROTOCOL-STATUSFILE*    "${LISP_PROTOCOL_STATUSFILE}")
(setq *AUTOLISP-PROTOCOL-STDINFILE*     "${LISP_PROTOCOL_STDINFILE}")
(setq *AUTOLISP-PROTOCOL-STDOUTFILE*    "${LISP_PROTOCOL_STDOUTFILE}")
(setq *AUTOLISP-PROTOCOL-STDERRFILE*    "${LISP_PROTOCOL_STDERRFILE}")
(setq *AUTOLISP-PROTOCOL-CONTROLFILE*   "${LISP_PROTOCOL_CONTROLFILE}")
(setq *AUTOLISP-PROTOCOL-HEARTBEATFILE* "${LISP_PROTOCOL_HEARTBEATFILE}")
(setq *AUTOLISP-PROTOCOL-READFILE*      "${LISP_PROTOCOL_READFILE}")
(setq *AUTOLISP-PROTOCOL-INFOFILE*      "${LISP_PROTOCOL_INFOFILE}")
(setq *AUTOLISP-PROTOCOL-RUNTIMEFILE*   "${LISP_PROTOCOL_RUNTIME_FILE}")
(setq *AUTOLISP-USE-REMOTE-PROTOCOL*    ${USE_REMOTE_PROTOCOL})
(setq *AUTOLISP-LOGNAME*                "autolisp-session.log")
(setq *AUTOLISP-QUIT-ON-FINISH*         ${QUIT_ON_FINISH})
(setq *AUTOLISP-BOOTSTRAP-PHASE*        "${BOOTSTRAP_PHASE}")
(setq *AUTOLISP-INVOCATION-DIR*         "${INVOCATION_DIR}")
(setq *AUTOLISP-DEBUG*                  ${T_or_nil})
(setq *AUTOLISP-DEBUGFILE*              "${DEBUGFILE}")
(setq *AUTOLISP-VERSION*                '(${MAJOR} ${MINOR} ${DEVELOP}))

;; --- Path helpers (autolisp:2497-2561) ---
(defun autolisp-absolute-path-p (p)
  (cond
    ((or (null p) (not (= (type p) 'STR)) (= p "")) nil)
    ((or (= (substr p 1 1) "/")
```

```
         (= (substr p 1 1) "\\")) T)
    ((and (>= (strlen p) 2)
          (= (substr p 2 1) ":")) T)
    (T nil)))

(defun autolisp-resolve-path (p)
  (cond
    ((or (null p) (not (= (type p) 'STR))) p)
    ((autolisp-absolute-path-p p) p)
    ((or (null *AUTOLISP-INVOCATION-DIR*) (= *AUTOLISP-INVOCATION-DIR* "")) p)
    (T (strcat *AUTOLISP-INVOCATION-DIR* "/" p))))

(defun autolisp-runtime-debug (msg)
  (if *AUTOLISP-DEBUG*
    (autolisp-bootstrap-append-line *AUTOLISP-DEBUGFILE* msg)))

;; --- I/O write helpers (autolisp:2563-2636) ---
(defun autolisp-bootstrap-append-line (path text / f)
  (if (and (= (type path) 'STR) (/= path ""))
    (progn
      (setq f (open path "a"))
      (if f (progn (write-line text f) (close f))))))

(defun autolisp-bootstrap-write-line (path text / f)
  (if (and (= (type path) 'STR) (/= path ""))
    (progn
      (setq f (open path "w"))
      (if f (progn (write-line text f) (close f))))))

(defun autolisp-bootstrap-mark-fatal (phase err / msg)
  (setq msg (autolisp-bootstrap-render-error err))
  (autolisp-bootstrap-append-line *AUTOLISP-ERRFILE*
                                  (strcat "ERROR " phase ": " msg))
  (autolisp-bootstrap-write-line *AUTOLISP-STATUSFILE* "1")
  (autolisp-bootstrap-write-line *AUTOLISP-PROTOCOL-STATUSFILE*
                                 (strcat "FAILED " phase))
  (if *AUTOLISP-QUIT-ON-FINISH*
    (progn
      (vl-catch-all-apply 'setvar (list "FILEDIA" 0))
      (vl-catch-all-apply 'command (list "_QUIT" "_N")))))
```

```
  1)

;; --- User helpers (autolisp:2638-2747) ---
(defun autolisp-set-status (code / f)
  (setq f (open *AUTOLISP-STATUSFILE* "w"))
  (if f (progn (write-line (itoa code) f) (close f)))
  code)

(defun autolisp-set-status-text (text / f)
  (setq f (open *AUTOLISP-STATUSFILE* "w"))
  (if f (progn (write-line text f) (close f)))
  text)

(defun autolisp-log-out (text)
  (autolisp-write-line *AUTOLISP-OUTFILE* text))

(defun autolisp-log-err (text)
  (autolisp-write-line *AUTOLISP-ERRFILE* text))

;; --- Quit signal (autolisp:~3290) ---
(setq *AUTOLISP-QUIT-SIGNAL* "--AUTOLISP-QUIT-SIGNAL--")
(setq *AUTOLISP-QUIT-REQUESTED* nil)

(defun autolisp-quit-signal-p (msg)
  (and (= (type msg) 'STR)
       (= msg *AUTOLISP-QUIT-SIGNAL*)))

(defun quit () (setq *AUTOLISP-QUIT-REQUESTED* T)
               (error *AUTOLISP-QUIT-SIGNAL*))
(defun exit () (quit))
(defun autolisp-exit () nil)         ; leave REPL without engine quit

;; --- LOG phase (autolisp:3324-3367) ---
(defun autolisp-log-setup (/ old-mode old-path old-name)
  (setq old-mode (autolisp-safe-getvar "LOGFILEMODE"))
  (setq old-path (autolisp-safe-getvar "LOGFILEPATH"))
  (setq old-name (autolisp-safe-getvar "LOGFILENAME"))
  (autolisp-try-setvar "LOGFILEPATH" *AUTOLISP-LOGDIR*)
  (if *AUTOLISP-LOGNAME*
    (autolisp-try-setvar "LOGFILENAME" *AUTOLISP-LOGNAME*))
```

```
  (autolisp-try-setvar "LOGFILEMODE" 1)
  (list old-mode old-path old-name))

;; --- FULL phase: output capture (autolisp:3369-3506) ---
(defun print (obj)
  (autolisp-princ-newline)
  (autolisp-emit-user-out obj))

(defun princ (obj)
  (autolisp-emit-user-line (autolisp-str obj))
  obj)

(defun prin1 (obj)
  (autolisp-emit-user-out obj))

(defun prompt (msg)
  (if msg (progn (autolisp-emit-user-line (autolisp-str msg)) msg) msg))

;; --- Historical batch REPL loop (used when no remote protocol) ---
(defun autolisp-repl-loop (/ keep-going)
  (autolisp-set-status-text "READY 0")
  (setq keep-going T)
  (while keep-going
    (while (not (findfile *AUTOLISP-INPFILE*))
      (autolisp-sleep-ms 100))
    (if (not (autolisp-run-repl-request))
      (setq keep-going nil)))
  (if *AUTOLISP-QUIT-ON-FINISH*
    (command "_QUIT" "_Y")))

;; --- Per-form request evaluation ---
(defun autolisp-eval-request-form (form)
  (setq form (autolisp-normalize-princ-call form))
  (if (autolisp-load-form-p form)
    (autolisp-eval-load-form form)
    (eval form)))

;; --- Main entry point (autolisp:3508-3562) ---
(autolisp-write-runtime-info)
```

```
;; Three alternative variants depending on the mode (the wrapper
;; emits exactly ONE, selected by USE-REMOTE-PROTOCOL / INTERACTIVE
;; / ENGINE):

;; Variant A: remote protocol
(defun autolisp-main-entry ()
  (load *AUTOLISP-PROTOCOL-RUNTIMEFILE*)
  (autolisp-protocol-server-loop))

;; Variant B: interactive batch REPL (BricsCAD macOS batch -i)
(defun autolisp-main-entry ()
  (autolisp-repl-loop))

;; Variant C: sequential execution of the actions
(defun autolisp-main-entry ()
  (vl-catch-all-apply 'autolisp-install-vlisp-compile-shadow nil)
  (if (autolisp-run-load 1 "${abs_src_1}")
    (autolisp-note-ok) (autolisp-note-fail))
  ;;  one line per action
  (if (= *AUTOLISP-FAIL* 0)
    (if (autolisp-run-main 2 "${MAIN}")
      (autolisp-note-ok) (autolisp-note-fail)))
  (autolisp-log-out (autolisp-summary-line))
  (if (> *AUTOLISP-FAIL* 0)
    (autolisp-finish 1)
    (autolisp-finish 0)))

;; --- Bootstrap finalisation (autolisp:3557-3562) ---
(setq _autolisp_main_result (vl-catch-all-apply 'autolisp-main-entry nil))
(if (vl-catch-all-error-p _autolisp_main_result)
  (autolisp-bootstrap-mark-fatal "bootstrap" _autolisp_main_result)
  _autolisp_main_result)
```

3.6 Status vocabulary

The numeric statuses written to `STATUSFILE`:

Code	Meaning
`0`	Final success.
`1`	Failure.
`99`	Initialisation marker (transient).

The textual statuses written to `*AUTOLISP-PROTOCOL-STATUSFILE*` (remote protocol):

Text status	Meaning
`BOOTING`	The runtime is starting up.
`READY 0`	Initialised, waiting.
`READY N`	Request N processed; ready for the next.
`RUNNING N`	Currently processing request N.
`DONE N OK`	Request N completed successfully.
`DONE N FAIL`	Request N completed in failure.
`DONE N QUIT`	Request N triggered a shutdown request.
`STOPPING`	Shutdown requested and accepted.
`STOPPED`	Runtime loop terminated.
`WAITING-INPUT`	The runtime is waiting for an input line.
`FAILED <phase>`	Fatal error during the named phase.
`FAILED READ`	Fatal error during a protocol read.

4 Engine selection and backend Œ system matrix

4.1 Default algorithm

```
if --bricscad, --autocad or --clautolisp is explicit:
    use that backend
else if MS-Windows:
    if AUTOCAD_EXE or AUTOCAD_ACCORECONSOLE or
       glob "C:/Program Files/Autodesk/*/accoreconsole.exe":
        backend = autocad
    else if BRICSCAD_EXE or
            glob "C:/Program Files*/Bricsys/*/bricscad.exe":
        backend = bricscad
    else:
        backend = clautolisp     ; new: in-process fallback
else (macOS, Linux):
    if /Applications/BricsCAD*.app (macOS) or BRICSCAD_EXE:
        backend = bricscad
    else:
        backend = clautolisp     ; new: in-process fallback
```

The historical "no CAD `> exit 3`" fallback is replaced by the `=clautolisp` backend, which makes the tool usable without any CAD installation.

4.2 Supported matrix

Backend	macOS	Linux	MS-Windows
`bricscad`	`batch` ; `automation` via AppleScript	Not supportedź	`automation` via COM (def
`autocad`	Not supportedš	Not supportedš	`automation` via COM (def
`clautolisp`	(default) Always supported	Always supported	Always supported

ź Bricsys distributes BricsCAD for Linux, but the historical wrapper did not support it. The new implementation must reuse the same pattern as macOS batch (`bricscad -b run.scr`), adjusting paths, and without an AppleScript bridge.

š Autodesk distributes neither AutoCAD nor `accoreconsole` for macOS or Linux. The `--autocad` combination on those systems fails cleanly with code `3`.

5 Backend bricscad

5.1 macOS

5.1.1 Bundle discovery

The algorithm searches, in order:

1. `$BRICSCAD_EXE` if the variable points at an existing file;
2. the first match of `/Applications/BricsCAD*.app`;
3. failing that, exit `3`.

The actual executable is `$app/Contents/MacOS/bricscad`.

Implementation reference: `autolisp:16611679` (`find_macos_bricscad_app`, `find_macos_bricscad_cli`).

5.1.2 Template discovery

The wrapper needs a DWG/DWT file to open in order to avoid the *New drawing* dialog. Search order:

1. `$AUTOLISP_BRICSCAD_TEMPLATE`;
2. `--dwg FILE` / `$DWG`;
3. `~/Library/Application Support/Bricsys/BricsCAD/V*/en_US/Templates/Default-mm.dwt`

4. ... same with `Default-m.dwt`, then without the `en_US/` subdirectory;

5. same paths under `/Library/Application Support/...` (system-wide).

Reference: `autolisp:16811716` (`find_macos_bricscad_template`).

5.1.3 `batch` mode

This is the default on macOS when `find_macos_bricscad_cli` succeeds.

Effective command:

```
/Applications/BricsCAD V26.app/Contents/MacOS/bricscad \
    <template.dwt> \
    [-P <BRICSCAD_MACOS_PROFILE>] \      ; default "lisp"
    -B <WORKDIR/run.scr>
```

The `run.scr` script emitted for macOS batch:

```
(load "<RUNLSPFILE>")
._FILEDIA 0
._QUIT _N
```

The last two lines are safeguards: should `(load )` fail before `autolisp-finish` is reached, `._FILEDIA 0` forces command-line prompts (avoiding any modal dialog), then `._QUIT _N` closes without saving.

PID detection: the wrapper polls `ps -axo pid,command==` looking for a process whose command line contains `$SCRFILE`. Reference: `autolisp:18631881` (`find_bricscad_batch_pid`).

Process monitoring: `kill -0 $pid` on the tracked PID; if the PID no longer responds, `find_bricscad_batch_pid` is rerun to find a potential re-instance. Reference: `autolisp:18831897`.

`_QUIT` quirk on macOS: `(command "_QUIT" "_Y")` on the AutoLISP side **does not** kill the BricsCAD process (macOS convention: the application stays alive without a document). After LISP has written its final status, the wrapper waits for the process to exit during `BRICSCAD_BATCH_EXIT_TIMEOUT_SECS` (default `10` s), then sends `SIGTERM`; if the process persists 1 second later, `SIGKILL`. Reference: `autolisp:641666` (`wait_for_batch_process_exit`) and `autolisp:45494588` (`kill_bricscad_batch_process`).

Workdir: as an exception to the default, macOS batch mode uses `${TMPDIR:-/tmp}/autolisp-runs` as the root.

5.1.4 `automation` mode

Selected when `--mode automation` is explicit, or when `find_macos_bricscad_cli` fails (no CLI binary found). `--backend attach` requires an already-running instance; `--backend launch` opens BricsCAD before injection.

The wrapper uses `osascript` to inject the `(load "...")` form into the active BricsCAD instance's command line. The AppleScript emitted verbatim:

```
on resetBricscadUI(appName, resetEscapes, resetDelay, commandDelay)
  tell application appName to activate
  tell application "System Events"
    tell process appName
      set frontmost to true
      if (count of windows) > 0 then
        try
          perform action "AXRaise" of window 1
        end try
      end if
    end tell
    delay commandDelay
    repeat resetEscapes times
      key code 53                    -- ESC
      delay resetDelay
    end repeat
    keystroke "_.COMMANDLINE"
    key code 36                      -- Return
    delay commandDelay
    repeat 2 times
      key code 53
      delay resetDelay
    end repeat
    delay commandDelay
  end tell
end resetBricscadUI

on run argv
  set appName to item 1 of argv
  set lispCmd to item 2 of argv
  set resetEscapes to (item 3 of argv) as integer
```

```
  set resetDelay to (item 4 of argv) as real
  set commandDelay to (item 5 of argv) as real

  delay 1
  resetBricscadUI(appName, resetEscapes, resetDelay, commandDelay)
  tell application "System Events"
    keystroke lispCmd
    key code 36
  end tell
end run
```

Reference: `autolisp:44094455`.

Checking that an instance is running:

```
on run argv
  set appName to item 1 of argv
  tell application "System Events"
    if exists process appName then
      return "1"
    end if
  end tell
  return "0"
end run
```

Reference: `autolisp:18991916` (`macos_app_running`).

Tuning parameters (env vars, all optional):

Variable	Default	Role
`AUTOLISP_MACOS_BRICSCAD_RESET_ESCAPES`	`6`	Number of ESC keystrokes in the *rese*
`AUTOLISP_MACOS_BRICSCAD_RESET_DELAY_SECS`	`0.25`	Delay between successive ESC keystro
`AUTOLISP_MACOS_BRICSCAD_COMMAND_DELAY_SECS`	`0.4`	Delay before and after command injec

Known restrictions:

- *Accessibility* permission is required for the `osascript` process (or for the host terminal).
- The `2D Drafting (Modern)` workspace may block injection; the classic `2D Drafting` workspace is recommended.
- The `_.COMMANDLINE` sequence flips the BricsCAD window into command-line mode; the user may see this happen.

5.1.5 Cleanup and signals

Trap `EXIT`: `rm -rf $WORKDIR` unless `CLEANUP_WORKDIR=1`. Trap `INT TERM`: sets `CLEANUP_WORKDIR=1`, keeps the workdir, kills the BricsCAD batch processes and the COM bridges, and completes the handshake in flight. Reference: `autolisp:16141637`.

5.2 Linux

Not supported by the historical wrapper. The new implementation must, on Linux:

- look up `$BRICSCAD_EXE` then a `bricscad` binary on `$PATH` (Bricsys installs by default in `/opt/bricsys/bricscad/V<N>/bricscad`);
- use the same pattern as macOS batch: `bricscad <template.dwg> -B run.scr`;
- handle the `automation` mode via D-Bus or via batch mode plus a REPL loop (depending on what BricsCAD Linux supports).

The lack of historical support means no operational template is documented; this must be worked out during implementation.

5.3 MS-Windows

5.3.1 Executable discovery

Variable	Effect
`BRICSCAD_EXE`	Explicit path to `bricscad.exe`.

Failing that, glob `/c/Program Files*/Bricsys/*/bricscad.exe`, in descending lexicographic order (newest first).

5.3.2 `automation` mode (COM, default)

The wrapper generates `bridge-bricscad.vbs` and launches it via:

```
cscript //nologo <WORKDIR>/bridge-bricscad.vbs \
    >"<WORKDIR>/bricscad-com-stdout.txt" \
    2>"<WORKDIR>/bricscad-com-stderr.txt"
```

COM mode is selected by `BRICSCAD_COM_MODE`:

Value	Meaning
`auto`	Try to attach an instance; launch a new one otherwise. (default.)
`attach`	Fail if no instance is open.
`launch`	Always launch a new instance.
`off`	Disable COM; switch to `batch` mode.

The `--mode` option overrides `BRICSCAD_COM_MODE` (and its `AUTOCAD_COM_MODE` equivalent) globally.

Verbatim VBScript (`bridge-bricscad.vbs`):

```
Option Explicit
Dim fso, shell, mode, exePath, runLsp, statusFile, outFile, errFile
Dim app, doc, cmd, created, attached, rc
Set fso = CreateObject("Scripting.FileSystemObject")
Set shell = CreateObject("WScript.Shell")
mode = "<MODE>"
exePath = "<WIN_BRICSCAD_EXE>"
runLsp = "<WIN_RUNLSP>"
statusFile = "<WIN_STATUSFILE>"
outFile = "<WIN_OUTFILE>"
errFile = "<WIN_ERRFILE>"
created = False
attached = False
rc = 0

Sub AppendLine(path, text)
  Dim f
  Set f = fso.OpenTextFile(path, 8, True)
  f.WriteLine text
  f.Close
End Sub

Function TryGetApp()
  On Error Resume Next
  Set TryGetApp = GetObject(, "BricscadApp.AcadApplication")
  If Err.Number <> 0 Then
    Err.Clear
    Set TryGetApp = Nothing
  End If
```

```
  On Error GoTo 0
End Function

Function TryCreateApp()
  On Error Resume Next
  If exePath <> "" Then
    shell.Environment("PROCESS")("BRICSCAD_LOCATION") = exePath
  End If
  Set TryCreateApp = CreateObject("BricscadApp.AcadApplication")
  If Err.Number <> 0 Then
    Err.Clear
    Set TryCreateApp = Nothing
  End If
  On Error GoTo 0
End Function

Set app = Nothing
If mode = "attach" Or mode = "auto" Then
  Set app = TryGetApp()
  If Not app Is Nothing Then attached = True
End If

If app Is Nothing Then
  If mode = "attach" Then
    AppendLine errFile, "ERROR COM attach: no running BricsCAD instance found."
    rc = 2
    WScript.Echo "ATTACHED=0"
    WScript.Echo "CREATED=0"
    WScript.Quit rc
  End If
  If mode = "launch" Or mode = "auto" Then
    Set app = TryCreateApp()
    If app Is Nothing Then
      AppendLine errFile, "ERROR COM launch: unable to create BricsCAD COM applicatio
      rc = 3
      WScript.Echo "ATTACHED=0"
      WScript.Echo "CREATED=0"
      WScript.Quit rc
    End If
    created = True
```

```
  End If
End If

If app Is Nothing Then
  AppendLine errFile, "ERROR COM bridge: unsupported mode or COM unavailable."
  rc = 4
  WScript.Echo "ATTACHED=0"
  WScript.Echo "CREATED=0"
  WScript.Quit rc
End If

On Error Resume Next
app.Visible = True
If Err.Number <> 0 Then Err.Clear
Set doc = app.ActiveDocument
If Err.Number <> 0 Then
  Err.Clear
  Set doc = Nothing
End If
If doc Is Nothing Then
  Set doc = app.Documents.Add
  If Err.Number <> 0 Then
    AppendLine errFile, "ERROR COM bridge: unable to get or create ActiveDocument."
    Err.Clear
    rc = 5
    If attached Then
      WScript.Echo "ATTACHED=1"
    Else
      WScript.Echo "ATTACHED=0"
    End If
    If created Then
      WScript.Echo "CREATED=1"
    Else
      WScript.Echo "CREATED=0"
    End If
    WScript.Quit rc
  End If
End If
cmd = "(load """ & Replace(runLsp, "\", "/") & """)" & vbCr
Err.Clear
```

```
Call doc.SendCommand(cmd)
If Err.Number <> 0 Then
  AppendLine errFile, "ERROR COM bridge: SendCommand failed: " & Err.Description
  Err.Clear
  rc = 6
Else
  rc = 0
End If
On Error GoTo 0
If attached Then
  WScript.Echo "ATTACHED=1"
Else
  WScript.Echo "ATTACHED=0"
End If
If created Then
  WScript.Echo "CREATED=1"
Else
  WScript.Echo "CREATED=0"
End If
WScript.Quit rc
```

Notable differences from the AutoCAD bridge:

- ProgID `BricscadApp.AcadApplication` (vs `AutoCAD.Application`).
- Environment variable passed to the engine: `BRICSCAD_LOCATION` (vs `AUTOCAD_LOCATION`).
- No `WaitQuiescent` loop: `SendCommand` is considered synchronous on the BricsCAD side; the bridge exits immediately and the wrapper then polls `STATUSFILE`.
- No EPURE handling: when `--epure` is requested under BricsCAD, the wrapper forces `BRICSCAD_COM_MODE=off` and falls back to batch mode.
- No debug file and no `VBSDebug` sub.

Bridge exit codes:

Code	Cause
0	Success.
2	`mode=attach` and no instance found.
3	`CreateObject` failed.
4	COM mode unsupported or unavailable.
5	Unable to get or create `ActiveDocument`.
6	`SendCommand` raised an error.

The bridge writes two lines on `stdout`, `ATTACHED=0|1` and `CREATED=0|1`, which the wrapper parses to decide cleanup.

5.3.3 `batch` mode

Selected by `--mode batch` or `BRICSCAD_COM_MODE=off`. Launches `bricscad.exe` with `-B <run.scr>`, equivalent to the macOS batch mode. The emitted `run.scr` is minimal:

```
(load "<RUNLSPFILE>")
```

The `._FILEDIA 0` / `._QUIT _N` safeguards are emitted only on macOS, because Windows `bricscad.exe` honours `(command "_QUIT" "_Y")`.

6 Backend autocad

6.1 macOS and Linux

AutoCAD is not distributed for these systems. `--autocad` there exits cleanly with code `3` and an error message.

6.2 MS-Windows

6.2.1 Executable discovery

Variable	Effect
`AUTOCAD_EXE`	Explicit path to `acad.exe` or `acadlt.exe`.
`AUTOCAD_ACCORECONSOLE`	Explicit path to `accoreconsole.exe` (`batch` mode only).

Failing that, glob:

1. `/c/Program Files/Autodesk/AutoCAD\ */acad.exe`, descending sort;

2. `/c/Program Files/Autodesk/AutoCAD\ LT\ */acadlt.exe`, descending sort.

Reference: `autolisp:13951405`.

6.2.2 `automation` mode (COM, default)

Required for AutoLISP functions that are not available inside `accoreconsole.exe` (the headless Core Console), most notably `vlisp-compile` which produces `.vlx` files: `vlide.dll` and `vl.arx` are loaded inside `acad.exe` and there is no standalone `vlide.exe`, so only a full instance can execute `vlisp-compile`.

The wrapper generates `bridge-autocad.vbs` and launches it via:

```
cscript //nologo <WORKDIR>/bridge-autocad.vbs \
    >"<WORKDIR>/autocad-com-stdout.txt" \
    2>"<WORKDIR>/autocad-com-stderr.txt"
```

COM mode is selected by `AUTOCAD_COM_MODE`: values `auto` (default), `attach`, `launch`, `off`. The `--mode` option takes priority.

Verbatim VBScript (`bridge-autocad.vbs`):

```
Option Explicit
Dim fso, shell, mode, exePath, runLsp, errFile, statusFile, debugFile, debugFlag, inv
Dim app, doc, cmd, progid, created, attached, rc
Dim waitSecs, waited, statusText
Set fso = CreateObject("Scripting.FileSystemObject")
Set shell = CreateObject("WScript.Shell")
mode = "<AUTOCAD_COM_MODE>"
exePath = "<WIN_AUTOCAD_EXE>"
runLsp = "<WIN_RUNLSP>"
errFile = "<WIN_ERRFILE>"
statusFile = "<WIN_STATUSFILE>"
debugFile = "<WIN_DEBUGFILE>"
debugFlag = "<AUTOLISP_DEBUG>"
invocationDir = "<WIN_INVOCATION_DIR>"
epureScript = "<WIN_EPURE_SCR>"
epureRequested = "<EPURE_REQUESTED>"

Sub VBSDebug(msg)
  Dim f
  If debugFlag <> "1" Then Exit Sub
```

```
  On Error Resume Next
  Set f = fso.OpenTextFile(debugFile, 8, True)
  If Err.Number = 0 Then
    f.WriteLine "[VBS] " & msg
    f.Close
  End If
  Err.Clear
  On Error GoTo 0
End Sub

VBSDebug "bridge start mode=" & mode & " exePath=" & exePath

' Align the VBScript process cwd to the one from which the user
' invoked alfe, BEFORE CreateObject. On Windows, an acad.exe
' out-of-process COM server launched via activation can inherit
' the activator's cwd in some contexts; even though that is not
' guaranteed, it helps with relative paths that appear in
' (load ...), findfile, etc.
If invocationDir <> "" Then
  On Error Resume Next
  If fso.FolderExists(invocationDir) Then
    shell.CurrentDirectory = invocationDir
  End If
  Err.Clear
  On Error GoTo 0
End If
waitSecs = CLng("<AUTOLISP_WAIT_SECS>")
If waitSecs < 5 Then waitSecs = 5
progid = "AutoCAD.Application"
created = False
attached = False
rc = 0

Sub AppendLine(path, text)
  Dim f
  Set f = fso.OpenTextFile(path, 8, True)
  f.WriteLine text
  f.Close
End Sub
```

```
Sub EmitFlags(att, cre)
  Dim a, c
  If att Then a = "1" Else a = "0"
  If cre Then c = "1" Else c = "0"
  WScript.StdOut.WriteLine "ATTACHED=" & a
  WScript.StdOut.WriteLine "CREATED=" & c
End Sub

Function ReadStatusText(path)
  Dim f, s
  ReadStatusText = ""
  If Not fso.FileExists(path) Then Exit Function
  On Error Resume Next
  Set f = fso.OpenTextFile(path, 1, False)
  If Err.Number <> 0 Then
    Err.Clear
    Exit Function
  End If
  If Not f.AtEndOfStream Then
    s = f.ReadAll
  Else
    s = ""
  End If
  f.Close
  On Error GoTo 0
  ReadStatusText = Trim(s)
End Function

Function StatusReady(path)
  Dim s
  s = ReadStatusText(path)
  If s = "" Then
    StatusReady = False
  ElseIf s = "__PENDING__" Then
    StatusReady = False
  ElseIf s = "99" Then
    StatusReady = False
  Else
    StatusReady = True
  End If
```

```
End Function

Sub WaitQuiescent(a, secs)
  Dim deadline, state, quiescent, supported
  deadline = Timer + secs
  supported = True
  ' Very short initial sleep: let any splash/profile pass before
  ' the first IsQuiescent query.
  WScript.Sleep 200
  Do
    quiescent = False
    On Error Resume Next
    Set state = a.GetAcadState
    If Err.Number <> 0 Then
      Err.Clear
      supported = False
      Exit Do
    End If
    quiescent = state.IsQuiescent
    If Err.Number <> 0 Then
      Err.Clear
      supported = False
      Exit Do
    End If
    On Error GoTo 0
    If quiescent Then Exit Sub
    WScript.Sleep 500
  Loop While Timer < deadline
  On Error GoTo 0
  ' If IsQuiescent is not usable, fall back to a short passive wait.
  If Not supported Then WScript.Sleep 500
End Sub

Sub WaitStatus(path, secs)
  Dim deadline, lastStatus, currentStatus
  deadline = Timer + secs
  lastStatus = ""
  VBSDebug "WaitStatus begin secs=" & secs
  Do
    currentStatus = ReadStatusText(path)
```

```
    If currentStatus <> lastStatus Then
      VBSDebug "WaitStatus status='" & currentStatus & "'"
      lastStatus = currentStatus
    End If
    If StatusReady(path) Then
      VBSDebug "WaitStatus ready"
      Exit Sub
    End If
    WScript.Sleep 500
  Loop While Timer < deadline
  VBSDebug "WaitStatus timeout"
End Sub

Function AttachApp()
  On Error Resume Next
  Set AttachApp = GetObject(, progid)
  If Err.Number <> 0 Then
    Err.Clear
    Set AttachApp = Nothing
  End If
  On Error GoTo 0
End Function

Function CreateApp()
  On Error Resume Next
  If exePath <> "" Then
    shell.Environment("PROCESS")("AUTOCAD_LOCATION") = exePath
  End If
  Set CreateApp = CreateObject(progid)
  If Err.Number <> 0 Then
    Err.Clear
    Set CreateApp = Nothing
  End If
  On Error GoTo 0
End Function

' Explicitly initialise app/doc: otherwise these variants stay
' Empty when mode = "launch" (the attach/auto block does not Set
' them), and "If app Is Nothing" raises "Object required" because
' Empty is not an object.
```

```
Set app = Nothing
Set doc = Nothing

If mode = "attach" Or mode = "auto" Then
  VBSDebug "trying GetObject AttachApp"
  Set app = AttachApp()
  If Not app Is Nothing Then
    attached = True
    VBSDebug "attached to existing instance"
  End If
End If

If app Is Nothing Then
  If mode = "attach" Then
    AppendLine errFile, "ERROR COM attach: no running AutoCAD instance found."
    VBSDebug "attach failed: no running instance"
    EmitFlags False, False
    WScript.Quit 2
  End If
  If mode = "launch" Or mode = "auto" Then
    VBSDebug "calling CreateObject (this can take 10-30s)"
    Set app = CreateApp()
    If app Is Nothing Then
      AppendLine errFile, "ERROR COM launch: unable to create AutoCAD COM application
      VBSDebug "CreateObject failed"
      EmitFlags False, False
      WScript.Quit 3
    End If
    created = True
    VBSDebug "CreateObject ok, acad.exe should be running"
  End If
End If

If app Is Nothing Then
  AppendLine errFile, "ERROR COM bridge: unsupported AutoCAD COM mode."
  EmitFlags False, False
  WScript.Quit 4
End If

On Error Resume Next
```

```
app.Visible = True
If Err.Number <> 0 Then Err.Clear
On Error GoTo 0

If epureRequested = "1" Then
  On Error Resume Next
  app.Preferences.Profiles.ActiveProfile = "Epure"
  If Err.Number <> 0 Then
    AppendLine errFile, "WARN COM bridge: cannot activate AutoCAD profile 'Epure' ("
    Err.Clear
  End If
  On Error GoTo 0
End If

WaitQuiescent app, waitSecs

On Error Resume Next
Set doc = app.ActiveDocument
If Err.Number <> 0 Then
  Err.Clear
  Set doc = Nothing
End If
If doc Is Nothing Then
  Set doc = app.Documents.Add
  If Err.Number <> 0 Then
    AppendLine errFile, "ERROR COM bridge: unable to get or create AutoCAD ActiveDocu
    Err.Clear
    If created Then
      app.Quit
    End If
    EmitFlags attached, created
    WScript.Quit 5
  End If
End If
On Error GoTo 0

If epureScript <> "" Then
  cmd = "._SCRIPT " & Chr(34) & Replace(epureScript, "\", "/") & Chr(34) & vbCr
  On Error Resume Next
  Call doc.SendCommand(cmd)
```

```
  If Err.Number <> 0 Then
    AppendLine errFile, "WARN COM bridge: EPURE _SCRIPT send failed: " & Err.Descript
    Err.Clear
  End If
  On Error GoTo 0
End If

cmd = "(load """ & Replace(runLsp, "\", "/") & """)" & vbCr
VBSDebug "SendCommand " & cmd
On Error Resume Next
Call doc.SendCommand(cmd)
If Err.Number <> 0 Then
  AppendLine errFile, "ERROR COM bridge: AutoCAD SendCommand failed: " & Err.Descript
  VBSDebug "SendCommand failed: " & Err.Description
  Err.Clear
  rc = 6
End If
On Error GoTo 0

' SendCommand is asynchronous: wait until the LISP runtime has
' written STATUSFILE before considering app.Quit (otherwise we
' would kill acad.exe while it is still loading). We distinguish
' success (final status written) from timeout (status remains at
' __PENDING__/99): on timeout we do NOT call app.Quit, otherwise
' we would interrupt the LISP runtime still in flight. The bash
' wrapper detects this case and kills acad.exe via
' kill_autocad_com_process.
Dim statusReadyFlag
statusReadyFlag = False
If rc = 0 Then
  WaitStatus statusFile, waitSecs
  statusReadyFlag = StatusReady(statusFile)
End If

If created And statusReadyFlag Then
  VBSDebug "calling app.Quit (created=True, status ready)"
  On Error Resume Next
  app.Quit
  If Err.Number <> 0 Then Err.Clear
  On Error GoTo 0
```

```
End If

VBSDebug "bridge exit rc=" & rc & " attached=" & attached & " created=" & created
EmitFlags attached, created
WScript.Quit rc
```

Robustness applied by the bridge before user execution:

- wait on `app.GetAcadState.IsQuiescent` before the first `SendCommand`, capped at `AUTOLISP_WAIT_SECS` (default `180` s);
- explicit opening of a new document via `Documents.Add` when `ActiveDocument` is absent (`vlisp-compile` requires a document);
- wait on `STATUSFILE` before `app.Quit`, since `SendCommand` is asynchronous on the AutoCAD side;
- `app.Quit` conditioned on `created = True And statusReadyFlag` so that we neither close an attached instance owned by the user nor interrupt a still-running runtime.

Bridge exit codes:

Code	Cause
0	Success.
2	`mode=attach` and no instance found.
3	`CreateObject` failed.
4	COM mode unsupported.
5	Unable to get or create `ActiveDocument`.
6	`SendCommand` raised an error.

6.2.3 `batch` mode (`accoreconsole`)

Selected by `--mode batch` or `AUTOCAD_COM_MODE=off`.

Executable lookup:

```
local acc="${AUTOCAD_ACCORECONSOLE:-}"
if [[ -z "$acc" ]]; then
  acc="$(ls -1 "/c/Program Files/Autodesk"/*/accoreconsole.exe 2>/dev/null | head -n
fi
```

Prerequisites:

- a DWG: `--dwg FILE` or `$AUTOLISP_DWG`;
- a `.scr` file (LSP transformed into a script, at `$SCRFILE`).

Command:

```
"<acc>" /i "<dwg>" /s "<SCRFILE>" \
    >"<WORKDIR>/autocad-stdout.txt" \
    2>"<WORKDIR>/autocad-stderr.txt"
```

Flags:

- `/i <dwg>`: input drawing;
- `/s <scr>`: AutoCAD script or compiled LSP.

6.2.4 `vlisp-compile` use case

```
alfe --autocad --mode automation \
         -x '(vlisp-compile (quote lsa) "src-vlx/schmsplus.prj")' \
         --quit
```

`vlisp-compile` does not exist in `accoreconsole.exe`; `--mode automation` (and therefore the COM bridge) is required. The second argument selects the output format: `'st` for uncompressed `.fas`, `'lsa` for a compressed `.vlx` application.

6.2.5 VBS escaping helper

```
escape_vbs_string() {
  local s="$1"
  s=${s//\"/\"\"}
  printf '%s' "$s"
}
```

Doubling of quotes only (VBScript convention). Backslashes are not escaped: Windows paths flow through verbatim and `Replace(..., "\", "/")` is performed on the VBS side for paths passed to `(load )`.

7 Backend clautolisp

7.1 Overview

The `clautolisp` backend evaluates AutoLISP code **in-process** in the Common Lisp runtime provided by `clautolisp`. No CAD engine is launched. This is the default backend on every system when no CAD is detected, and it is usable without any CAD installation.

All other backends are meant to converge on this baseline: the evaluation of `-l` / `-x` actions goes through the same internal structures (`run-autolisp-file`, `run-autolisp-string`); only the effective HAL changes (`NullHost`, `MockHost`, `LiveHost`).

7.2 `clautolisp` modules used

Module	Role in `autolisp-front-end`
`autolisp-reader`	Source reading and form completeness for the REPL.
`autolisp-runtime`	Evaluator, contexts, environment, structured error handling.
`autolisp-builtins-core`	Builtins (numeric, list, string, file, stream).
`autolisp-host`	HAL: backend contract.
`autolisp-mock-host`	Mock host for `--host mock` (the default).

7.3 Dialect

`--dialect`	Meaning
`strict`	Strict reader, runtime conforming to the local `clautolisp` spec.
`autocad-2026`	AutoCAD 2026 dialect (read tolerances, builtin semantics).
`bricscad-v26`	BricsCAD V26 dialect.
`clautolisp`	Clautolisp dialect (with its specific extensions).

When the explicit backend is `bricscad`, the default dialect is `bricscad-v26`; for `autocad`, `autocad-2026`; for `clautolisp`, `strict`.

7.4 Invocation mode

The wrapper runs in-process there is **no** file-IPC for this backend. Nonetheless, for symmetry with the CAD backends and to ease diagnostics, a workdir is created and the `output.txt` / `errors.txt` / `status.txt` channels are populated in parallel with the writes to `stdout` / `stderr`. The remote protocol is not enabled.

The `alfe` binary can:

- either load the `clautolisp/*` ASDF packages and call the runtime APIs in-process (recommended);
- or delegate to `clautolisp-sbcl` via `uiop:launch-program` and talk to it via stdin/stdout pipes (the *subprocess* mode, useful for isolating the engine or for running the engine under a different Lisp).

7.5 Encodings

Full support via `babel`: every coding and every line ending. `-e` sets the session-wide source-file encoding (applied to every load in the session, including nested `(load )` calls a user init file makes); `-E` sets the I/O encoding.

7.5.1 Default source-file encoding precedence

When the caller does not pass `-e ENC`, `alfe` (and the `clautolisp` backend it drives) resolves the default encoding from the POSIX locale environment in the standard order:

1. `LC_ALL` global locale override.
2. `LC_CTYPE` character-classification + encoding category.
3. `LANG` catch-all default.

The encoding suffix of the first non-empty value wins (`en_US.UTF-8` `:utf-8`, `fr_FR.ISO-8859-1@euro :iso-8859-1`). A bare value with no `.ENCODING` suffix (`C`, `POSIX`, `en_US`) yields no override, and the resolution falls through to the dialect default (see `clautolisp` spec section *Encoding and line endings*: `:iso-8859-1` for `strict`, `:utf-8` for `autocad-2026` and `bricscad-v26`).

The OTHER `LC_*` categories (`LC_COLLATE`, `LC_MESSAGES`, `LC_NUMERIC`, `LC_TIME`,) are intentionally **not** consulted for source-file encoding. They govern string ordering, message translation, decimal separators, and date formats using them for byte-stream interpretation would be misuse of the POSIX standard.

The resulting full precedence is, strongest first:

Step	Source	Notes
1	`-e ENC` on the alfe command line	Explicit user intent.
2	`LC_ALL` / `LC_CTYPE` / `LANG`	POSIX host hint.
3	Dialect default	=strict=latin-1, autocad/bricscadutf-8.

The matching slot lives on the runtime session, so it applies to every load in the session including the nested `(load )` calls a user init file makes. The `subprocess` backend variant inherits the parent `alfe`'s environment, so the spawned `clautolisp-sbcl` performs its own equivalent probe and reaches the same conclusion.

7.6 macOS, Linux, MS-Windows

The backend is identical on the three systems; the only differences are:

- on Windows, paths normalised with `/` in the arguments passed to `(load )` (consistency with the other backends);
- `$TMPDIR` consulted as the workdir root (default `/tmp` on Unix, `%TEMP%` on Windows).

8 EPURE

Note: this option is specific to a single user. A way of defining such options as modular plug-ins should be found.

The `--epure` option is **Windows-only** and causes, before the main script runs:

- BricsCAD: launch with `/P Epure`, then load and execute `%APPDATA%\sncf\epure\epure 2022_b\control_path_epure_2022.scr`. When `--epure` is requested under BricsCAD, `BRICSCAD_COM_MODE` is forced to `off` (the COM bridge does not handle EPURE for BricsCAD).
- AutoCAD: `Epure` profile activated via `app.Preferences.Profiles.ActiveProfile`, then execution of `%APPDATA%\sncf\epure\epure 2022\control_path_epure_2022.scr` via `._SCRIPT`. `AUTOCAD_EXE` is required; `accoreconsole.exe` is excluded (no profile support).

Outside Windows, `--epure` is silently ignored (backward compatibility). A future evolution could support it under `clautolisp` if an equivalent of the EPURE software is available.

9 Remote REPL protocol

9.1 Purpose

Specifies the communication between:

- the shell client (`alfe`);
- the AutoLISP runtime embedded in `run-common.lsp` and loading `autolisp-remote-io.lsp`.

The central idea: everything goes through a single model of remote Lisp inputs/outputs, rather than separating *business requests* from *user inputs*. The shell becomes the *remote terminal* of the Lisp runtime.

9.2 Principles

- File-only communication.
- Synchronous protocol, driven by an explicit state.
- Shell writes to the input and control channels: atomic (`write to temp.X.tmp` then `mv`).
- AutoLISP writes to the output channels: ordered and deterministic.
- The reading primitive is the **line**, not the Lisp form.
- `read` is rebuilt on top of `read-line` and can consume several lines when the form is incomplete.

9.3 Protocol session directory

Under the workdir, the `protocol/` subdirectory contains the reserved files listed in the section.

9.4 Execution model

The shell launches the engine with a minimal script of the form:

```
._COMMANDLINE
(load "/.../run-common.lsp")
._QUIT _Y
```

The role of `run.scr` is deliberately minimal:

- prepare the command line;
- load `run-common.lsp`;
- quit the engine when `run-common.lsp` returns control.

The real protocol logic lives in:

- the shell (`lib/autolisp-remote-io.sh`);
- `run-common.lsp` (local helpers);
- `runtime/autolisp-remote-io.lsp` (loaded by `run-common.lsp` or by the shell passing it as an argument).

9.5 Roles

9.5.1 Shell

The shell:

- creates `WORKDIR/protocol/`;
- initialises the session files;
- launches the engine;
- writes remote input atomically into `stdin.txt`;
- writes out-of-band commands atomically into `control.txt`;
- reads `stdout.txt` and `stderr.txt` incrementally;
- waits for `status.txt` transitions;
- kills the engine process as a last resort.

9.5.2 AutoLISP runtime

The AutoLISP runtime:

- shadows some I/O functions;
- publishes its state into `status.txt`;
- reads incoming lines from `stdin.txt`;

- writes outputs into `stdout.txt` and `stderr.txt`;
- monitors `control.txt`;
- optionally maintains a `heartbeat.txt`;
- cleanly terminates the session when shutdown is requested.

9.6 Shadowed I/O functions

The protocol relies on shadowing the interactive or semi-interactive functions.

Priority targets:

- `read-line`
- `read`
- `princ`
- `print`
- `prin1`
- `prompt`

To consider next:

- `read-char`
- `getstring`
- `getint`
- `getreal`
- `getpoint` and other interactive CAD primitives, if they need to be supported.

Constraints:

- shadowing applies only to the remote interactive streams;
- explicit calls on a real file handle keep working as expected: `(read f)` must still read from `f`; `(read)` without an argument may be redirected to the remote input.

9.7 Reading model

Primitive: `autolisp-protocol-remote-read-line` waits for a line to become available in `stdin.txt`, consumes it, returns it.

`read` is not a single round-trip: it accumulates lines until syntactic balance (parens, strings, comments), then returns the read Lisp object. This naturally supports:

```
(loop (print (eval (read))))
```

9.8 States published in `status.txt`

Cf. table in the AutoLISP runtime-side contract section (status vocabulary).

9.9 `stdin.txt` channel

Carries the input lines provided by the shell.

- the shell writes into a temporary file (`stdin.txt.tmp.$$.RAND`);
- the shell publishes atomically via `mv` to `stdin.txt`;
- the runtime consumes the published lines, then deletes the file (`vl-file-delete`).

Consumption is destructive: once read, `stdin.txt` disappears until the next publication.

9.10 `control.txt` channel

Reserved for the out-of-band channel. Commands:

- `PING`: triggers a heartbeat beat;
- `SHUTDOWN`: requests a clean exit of the runtime loop, which publishes `STOPPING` then exits, then `STOPPED`;
- `INTERRUPT`: cooperative interrupt; sets `*AUTOLISP_PROTOCOL_INTERRUPT*`; cooperative user loops may consult this flag.

The runtime consumes these commands independently of the normal `stdin.txt` stream.

9.11 Heartbeat

`heartbeat.txt` is optional but recommended. The runtime writes a periodic timestamp there (`rtos (getvar "DATE") 2 8`). The shell thus distinguishes a live-but-idle runtime from a stuck one.

9.12 Server main loop

1. publish `BOOTING`;
2. initialise the remote I/O helpers;
3. publish `READY 0`;
4. read a form via `autolisp-protocol-remote-read`;
5. publish `RUNNING N`;
6. evaluate the form via `autolisp-eval-request-form`;
7. publish `DONE N OK` or `DONE N FAIL` or `DONE N QUIT`;
8. publish `READY N` and loop;
9. on `SHUTDOWN` or `(quit)`, publish `STOPPING` then exit, then `STOPPED`;
10. `run-common.lsp` returns control; `run.scr` resumes and runs `._QUIT _Y`.

Reference implementation: `autolisp-protocol-server-loop` in `runtime/autolisp-remote-io.l`

9.13 Shell helpers

Reference implementation: `lib/autolisp-remote-io.sh`, in particular:

- `autolisp_protocol_init_session WORKDIR`: creates `$WORKDIR/protocol/` and initialises the paths;
- `autolisp_protocol_export_session`: exports to the environment, for the runtime;
- `autolisp_protocol_reset_session`: resets the channels, publishes `BOOTING`;
- `autolisp_protocol_write_atomic_file target content`: writes to `target.tmp.$$.RAND` then `mv`;

- `autolisp_protocol_send_stdin TEXT`: waits for the `stdin.txt` slot to be free, then publishes;
- `autolisp_protocol_send_control TEXT`: same for `control.txt`;
- `autolisp_protocol_read_status`: reads the last non-empty line of `status.txt`;
- `autolisp_protocol_wait_for_status EXPECTED TIMEOUT` and `autolisp_protocol_wait_for_ PREFIX TIMEOUT`.

9.14 Interrupts

AutoLISP does not provide a generally usable mechanism for asynchronous preemption. Interruption is therefore cooperative:

- `autolisp-protocol-handle-control` reads `control.txt`, detects `INTERRUPT`, sets a flag;
- user loops may consult this flag and exit cleanly.

Non-interruptible cases: a long monolithic Lisp form, a non-cooperative user loop, a blocking CAD command. In those cases the shell must be able to observe the timeout, keep `WORKDIR`, and kill the engine process.

9.15 Safety invariants

- The shell never writes `stdin.txt` or `control.txt` without atomic publication.
- The runtime never processes partially-published data.
- The shell does not write new input if the runtime is not in a compatible state.
- Outputs are stable when the state transitions from `RUNNING` back to `READY`.
- Out-of-band commands stay separate from the normal input stream.

9.16 Target use cases

9.16.1 Remote REPL

The shell reads a user line, publishes it into `stdin.txt`, then displays the remote outputs.

9.16.2 Interactive `read`

`(read)` consumes as many remote lines as needed until the form is complete. Example: `42` returns the Lisp value `42`. An incomplete list requests continuations until balanced closure.

9.16.3 User-side mini Lisp REPL

```
(loop (print (eval (read))))
```

must work without a specific "business request" protocol.

9.17 Open questions

- per-character granularity via `read-char` (or stick strictly to line-by-line in a first pass);
- exact consumption format for `stdin.txt`;
- precise purge strategy for `control.txt`;
- expected compatibility level with CAD-specific interactive functions;
- cleanest way to distinguish `stdout`, `stderr`, and the *REPL result* from the shell UX side.

10 Environment variables

Consolidated list. The *CLI* column indicates the equivalent option when one exists.

10.1 Verbosity and debug

Variable	Values	CLI	Effect
`AUTOLISP_VERBOSE`	`1`	`--verbose`	Extended diagnostics.
`AUTOLISP_QUIET`	`1`	`--quiet`	Suppresses non-essential messages.
`AUTOLISP_DEBUG`	`1`	`--debug`	`[DEBUG]` traces and `runtime.debug.`
`AUTOLISP_DRY_RUN`	`1`	`--dry-run`	Does not execute the engine.
`AUTOLISP_KEEP_WORKDIR`	`1`		Keeps the workdir at end of executio
`AUTOLISP_SUPPRESS_VERSION_BANNER`	`1`		Suppresses the version banner (intern

10.2 Workdir and timeout

Variable	Values	CLI	Effect
AUTOLISP_WORKDIR	path		Workdir root.
AUTOLISP_WAIT_SECS	seconds	--timeout	Global engine-wait timeout
AUTOLISP_TIMEOUT	seconds		Old name, still accepted as
AUTOLISP_BRICSCAD_BATCH_STARTUP_TIMEOUT	seconds		macOS BricsCAD batch sta
AUTOLISP_BRICSCAD_BATCH_EXIT_TIMEOUT	seconds		Exit timeout after the final
TMPDIR	path		Alternative workdir root for

10.3 Backend and engine selection

Variable	Values	CLI	Effect
AUTOLISP_MODE	auto | automation | batch	--mode	CAD mode;
AUTOLISP_BACKEND	attach | launch	--backend	automation
AUTOLISP_OS	Darwin | Linux | MINGW64_NT-*		Overrides O
AUTOLISP_BOOTSTRAP_PHASE	marker | core | log | full	--bootstrap-phase	Bootstrap p
AUTOLISP_REMOTE_IO_MODE	auto | on | off		Enables/dis
AUTOLISP_DWG	path	--dwg	Default DW
AUTOLISP_EPURE	1	--epure	Activates --

10.4 Backend-specific

Variable	Values	Backend / System	Effect
AUTOCAD_EXE	path	autocad / Windows	Path to
AUTOCAD_ACCORECONSOLE	path	autocad / Windows	Path to
AUTOCAD_COM_MODE	auto | attach | launch | off	autocad / Windows	AutoCA
BRICSCAD_EXE	path	bricscad / all	Path to
BRICSCAD_COM_MODE	auto | attach | launch | off	bricscad / Windows	BricsCA
BRICSCAD_MACOS_PROFILE	name	bricscad / macOS batch	BricsCA
AUTOLISP_BRICSCAD_TEMPLATE	path	bricscad / macOS batch	Explicit

10.5 AppleScript tuning (bricscad macOS automation)

Variable	Default	Effect
AUTOLISP_MACOS_BRICSCAD_RESET_ESCAPES	6	Number of ESC keystrokes in the rese
AUTOLISP_MACOS_BRICSCAD_RESET_DELAY_SECS	0.25	Delay between successive ESCs.
AUTOLISP_MACOS_BRICSCAD_COMMAND_DELAY_SECS	0.4	Delay before/after command injection

10.6 Remote REPL protocol

Variable	Effect
`AUTOLISP_PROTOCOL_DIR`	protocol/ subdirectory.
`AUTOLISP_PROTOCOL_STATUSFILE`	(exported by the shell to the runtime; cf. `lib/autolisp-rem`
`AUTOLISP_PROTOCOL_STDINFILE`	same
`AUTOLISP_PROTOCOL_STDOUTFILE`	same
`AUTOLISP_PROTOCOL_STDERRFILE`	same
`AUTOLISP_PROTOCOL_CONTROLFILE`	same
`AUTOLISP_PROTOCOL_HEARTBEATFILE`	same
`AUTOLISP_PROTOCOL_READFILE`	same
`AUTOLISP_PROTOCOL_INFOFILE`	same; receives CAD-side product metadata.

10.7 Experimental

Variable	Effect
`AUTOLISP_EXPERIMENTAL_MACOS_BATCH_OPEN`	Enables an alternative launch path for BricsCAD ma

10.8 Init inhibition

Variable	CLI	Effect
`AUTOLISP_NO_INIT`	`--no-init=/`-norc=	Equivalent to `--no-init` for both alfe and clautolisp.
`ALFE_NO_INIT`	`--no-init=/`-norc=	Equivalent to `--no-init` for alfe only.
`CLAUTOLISP_NO_INIT`		Equivalent to `--no-init` for clautolisp only.

11 User init files

On startup, `alfe` walks a fixed four-slot lookup list and loads every existing file in order. Earlier files load before any user `-l` / `-x` / `--main` action, so the action queue runs against a fully-initialised evaluation context. The discovery + load- order policy is shared with the sibling `clautolisp` binary; both go through the `clautolisp.autolisp-init-files` module.

11.1 Stem list

#	Stem	Bare allowed?
1	`~/.autolisp{,rc}{,.lsp,.fas,.des,.vlx}`	yes
2	`~/.config/autolisp/init{,.lsp,.fas,.des,.vlx}`	no
3	`~/.alfe{,rc}{,.lsp,.fas,.des,.vlx}`	yes
4	`~/.config/alfe/init{,.lsp,.fas,.des,.vlx}`	no

Slots 1 and 2 are shared with the `clautolisp` binary so a user maintaining a single `~/.autolisp` file sees it loaded by both. Slots 3 and 4 are specific to alfe and run **after** the shared pair so program-specific definitions override the shared ones. Within each scope, the legacy HOME slot loads first and the XDG slot (`~/.config/<program>/init`) loads second, so a `~/.config/alfe/init.lsp` wins over both `~/.alfe` and the shared pair.

11.2 Per-stem extension preference

For each stem:

- The bare stem (without an extension) wins outright when the slot allows it (slots 1 and 3). XDG slots (2 and 4) ignore the bare stem.
- Otherwise the picker considers `<stem>.lsp` and the compiled variants `<stem>.{fas,des,vlx}`. A compiled variant wins only if strictly newer than `<stem>.lsp` (editing the source automatically retires a stale compile). Among qualifying compiled variants, the newest wins.
- Falls through to `<stem>.lsp` if no compiled variant qualified.
- The stem is silently skipped when no file matches.

11.3 Gating

The lookup is gated by:

- `--no-init` or `-norc` on the command line skip every stem.
- `$AUTOLISP_NO_INIT=1` (or `y` / `yes` / `true` / `on`, case-insensitive) shared kill-switch honoured by both binaries.
- `$ALFE_NO_INIT=1` disables init files for alfe only.

The CLI flag wins over the env vars; if it's present the env-var rung is not consulted.

11.4 Workdir cleanup

Mirroring the spec's `$AUTOLISP_KEEP_WORKDIR` env var, alfe honours `--keep-workdir` on the command line to skip the post-run workdir cleanup. The `--no-init` flag is *not* overloaded for this purpose (the V0 implementation accidentally tied the two slots together; the fix landed alongside the init-file work).

12 Versioning

- The historical bash wrapper `autolisp-script` version is read from `autolisp-script/VERSION.TXT` via `VERSION_MAJOR`, `VERSION_MINOR` and `VERSION_PATCH` (referenced *only* during the transition phase).
- The `autolisp-front-end` subproject follows the `MAJOR.MINOR.DEVELOP` convention already applied to `clautolisp` (cf. `tools/alfe/source/version.lisp`).
- `alfe --version` prints `alfe <MAJOR>.<MINOR>.<DEVELOP>` on `stdout` and exits `0`.
- Any modification to an `alfe` source or to a runtime dependency it loads automatically increments `DEVELOP`.

13 Tested invariants

- `-x` with and without visible output.
- `load` of a `.lsp` file; output produced during the `load`.
- `--main` with a custom entry point.
- `--interactive`: multi-line form, evaluation failure, remote REPL protocol.
- `--epure` mode under Windows for both CAD backends.
- `fake-cad` backend validating `run-common.lsp` generation and action sequencing without a real CAD (historical).
- `clautolisp --host mock` backend (new) covering `-x`, `-l`, `--main`, `--interactive` actions, and multi-line reading via the `autolisp-reader` reader.
- Encodings: round-trip `utf-8`, `iso-8859-1`, `windows-1252` with the three line terminations, for both `-l` and interactive I/O.

14 Known limitations and not-guaranteed behaviour

- The real Windows COM integration is exercised outside CI; its effective specification depends on the AutoCAD/BricsCAD installations.
- The macOS `automation` bootstrap depends on system Accessibility permissions and the current BricsCAD workspace.
- The detailed format of `--verbose` is liable to change; only the invariants covered by tests are normative.
- Interrupting a blocking CAD call can only be done by force (killing the process); the protocol does not allow preemptive interruption of AutoLISP code.
- The `bricscad` backend on Linux is to be worked out during implementation; no operational template is documented to date.
- `vlisp-compile` remains exclusive to `--autocad --mode automation` under Windows; no in-process equivalent is planned in `clautolisp` in the near term.

15 Appendices

15.1 A. Decision log

- bugs-closed/rewrite-in-common-lisp-decision.org: rewrite decision, scope, rationale, consequences on open items.

15.2 B. Reference files to reuse as-is

- `../runtime/autolisp-remote-io.lsp`: AutoLISP implementation of the protocol, to be ported to CL (local helpers) then re-emitted on the CAD side for the `bricscad` and `autocad` backends.
- `../lib/autolisp-remote-io.sh`: shell helpers of the protocol, to be ported to CL (driver side).

15.3 C. Mapping from bugs-open items to the new implementation

Item	Status
autolisp-encoding	Integrated in section ; native implementation via `babel` in the CL subproject.
kebab-case	Not applicable: convention applied by default by the new code.
identification-mark	Not applicable: `*AUTOLISP_VERSION*` defined by default by the new runtime's
final-comment	Covered by `autolisp-reader` in the CL subproject; a small fix on the bash sid